SIMATS ENGINEERING

ASSESSMENT TOOL 1: AI Model Design Exercise

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS

COURSE OUTCOME COVERED
COURSE CODE: MLA01

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.*(BTL 3)
Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 90 Mins No. of Questions: 1

Annexure A

AI Model Design Exercise

Code of Conduct:
I NAMBURI SAI YASHASWINI (192425250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: **yashaswini**

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Dataset Selection and Data Understanding	10%	
3. Data Preprocessing and Feature Engineering	10%	
4. AI Technique Selection and Justification	10%	
5. Model Design / Architecture	10%	
6. Algorithm / Pseudocode Development	5%	
7. Model Implementation and GitHub Repository	15%	
8. Experimental Design and Results	10%	
9. Performance Evaluation and Metrics	10%	
10. Result Analysis and Interpretation	5%	
11. Limitations and Future Enhancements	5%	
12. Documentation and Technical Presentation	10%	

Total	100%	

Signature of the Faculty Total Marks Awarded **ANSWER ANY ONE**

QUESTION

Handwritten Character Recognition Using a Multilayer Neural Network

1. Problem Statement

Handwritten character recognition is a classical pattern-recognition problem in which a system must automatically identify the digit or character represented by a scanned or photographed image of handwriting. This is a core capability behind postal-code readers, bank-cheque digit readers, digitized form processing, and handwriting-based input systems. The input to the system is a small grayscale image of a handwritten digit; the output is the predicted digit class (0-9). The AI task is a multi-class image classification problem, and the objective of this exercise is to design, implement, and evaluate a multilayer (feedforward) neural network that learns, through backpropagation, to map raw pixel intensities to the correct digit label.

Real-world handwriting varies significantly in stroke thickness, slant, size, and writer style, which makes rule-based recognition impractical. A neural network is well suited to this problem because it can automatically learn discriminative pixel-intensity patterns directly from labelled examples rather than relying on hand-crafted rules.

2. Objectives

- Design a multilayer feedforward neural network capable of classifying handwritten digit images into 10 classes (0-9).
- Preprocess raw pixel data through normalization and train/test splitting suitable for neural network training.
- Implement forward propagation and demonstrate how backpropagation updates network weights to minimize classification error.
- Train the network until convergence and evaluate it using standard classification metrics. •

Compare alternative network architectures and validate robustness using cross-validation.

- Analyse model behaviour, including sources of misclassification, and propose realistic improvements.

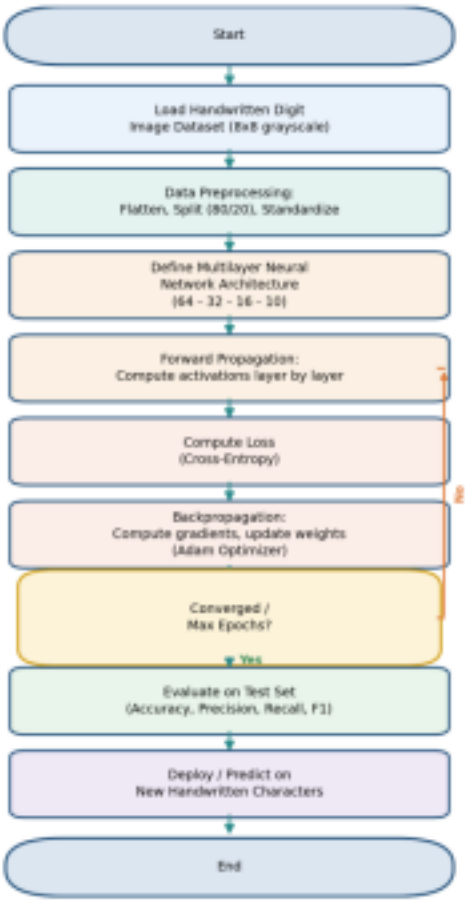

Figure 1: End-to-end methodology flowchart for the handwritten character recognition system.

3. Related Work / Literature Review

Handwritten digit recognition is one of the most extensively studied problems in pattern recognition and machine learning, largely because of the availability of standard benchmark datasets such as MNIST and the UCI/NIST digit collections used in this exercise. A brief overview of relevant approaches is summarized below.

Rumelhart, Hinton & Williams (1986) — Backpropagation	Introduced the generalized delta rule for training multilayer networks via gradient descent.	Foundation for all modern feedforward neural network training, including this exercise.
LeCun et al. (1998) — Convolutional Networks (LeNet)	Used local receptive fields and weight sharing to exploit spatial structure in digit images.	Higher accuracy on larger, noisier image datasets than fully connected networks.

k-Nearest Neighbours on pixel vectors	Classifies a digit by majority vote among its closest training examples in pixel space.	Simple, no training phase, but slow at prediction time and sensitive to noise.
Support Vector Machines with RBF kernel	Finds a maximum-margin non-linear decision boundary between digit classes.	Strong accuracy on small-to medium datasets, competitive with early neural networks.
This exercise — Multilayer Perceptron + Backpropagation	Fully connected 64-32-16-10 network trained with Adam-optimized backpropagation.	Lightweight, fast to train, and well matched to the small 8x8 digit dataset used here.

This exercise builds directly on the backpropagation approach, chosen over convolutional or kernel based alternatives because the 8x8 grayscale dataset is small enough that a fully connected multilayer perceptron can achieve high accuracy without the additional architectural or computational complexity of convolutional layers.

4. Dataset Description

The model is trained on the UCI/scikit-learn handwritten digits dataset (a widely used benchmark derived from the NIST database of handwritten digits). Each sample is an 8x8 grayscale image of a single handwritten digit, flattened into a 64-dimensional feature vector of pixel intensity values (0-16).

Source	scikit-learn built-in digits dataset (subset of the NIST handwritten digit database)
Total instances	1,797 labelled images
Image size	8 x 8 pixels, grayscale
Number of features	64 (flattened pixel intensity values, range 0-16)
Target variable	digit class, integer 0-9 (10 classes)
Data type	Numeric (integer pixel intensities); target is categorical (encoded as integer)
Class balance	Approximately balanced, ~178 samples per class
Train / Test split	1,437 training images (80%) / 360 test images (20%), stratified by class

5. Data Preprocessing

The following preprocessing pipeline was applied before model training:

- Flattening: each 8x8 image array is flattened into a 64-length feature vector.
- Train-test split: stratified 80/20 split (1,437 training samples, 360 test samples) preserving class proportions.
- Feature scaling: z-score standardization (zero mean, unit variance) fitted on the training set and applied to both train and test sets, using scikit-learn's StandardScaler — this is important for neural networks trained with gradient-based optimizers, since unscaled pixel values slow convergence.
- No missing values were present in this dataset, so imputation was not required.
- Feature selection: all 64 pixel features were retained, since each contributes spatial information; dimensionality reduction (e.g., PCA) was considered unnecessary given the dataset size and model capacity.

6. Selected AI Technique and Justification

Technique selected: Multilayer Feedforward Neural Network (Multilayer Perceptron) trained with the backpropagation algorithm.

Justification:

- Handwritten digit images contain non-linear, spatially distributed pixel patterns that are not linearly separable in raw pixel space — a multilayer network with non-linear (ReLU) activations can learn these non-linear decision boundaries, unlike a single-layer perceptron or plain linear classifier.
- Backpropagation provides an efficient, well-understood mechanism to compute the gradient of the loss with respect to every weight in the network, enabling gradient-based weight updates through multiple hidden layers.
- The dataset size (1,797 samples, 64 features) is well matched to a compact multilayer perceptron — large enough to generalize, small enough to avoid the need for convolutional layers or GPU-scale training.
- Compared with a decision tree or single statistical classifier, a neural network can model smoother, higher-capacity, non-linear decision surfaces, which is beneficial given the continuous-valued pixel intensity inputs.

7. Model Design / Architecture

The network is a fully connected, feedforward multilayer perceptron with two hidden layers. Each neuron in a layer is connected to every neuron in the subsequent layer (dense connectivity).

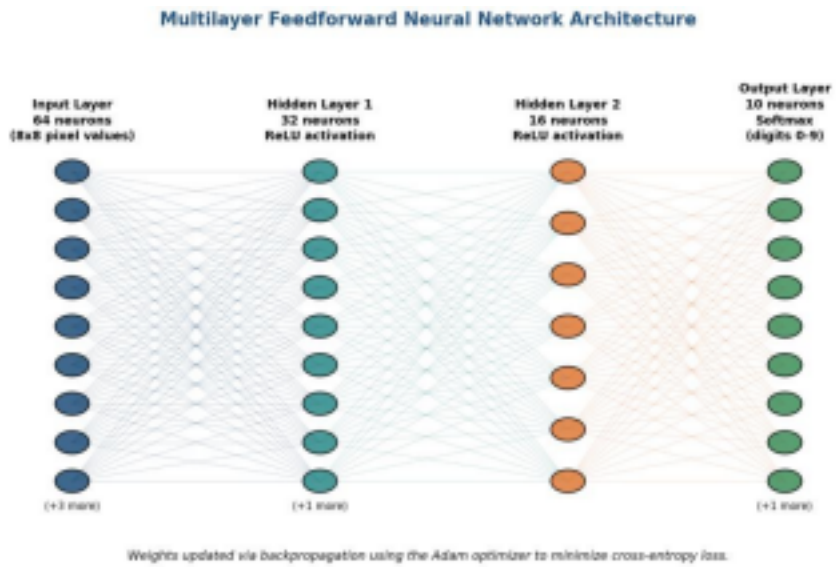

Figure 2: Multilayer neural network architecture (64 - 32 - 16 - 10).

Input layer	64 neurons (one per pixel feature)
Hidden layer 1	32 neurons, ReLU activation
Hidden layer 2	16 neurons, ReLU activation
Output layer	10 neurons, Softmax activation (one per digit class)
Loss function	Categorical cross-entropy
Optimizer	Adam (adaptive moment estimation)
Learning rate	0.001 (initial)
Regularization	L2 weight penalty, alpha = 1e-4
Max training epochs	400 (early stopping not required — see Section 10)
Weight initialization	Default scikit-learn initialization (scaled random values)

8. Algorithm / Pseudocode

Training procedure (forward propagation + backpropagation):

```
ALGORITHM: Train_MLP_Backpropagation
INPUT: X_train (n x 64 scaled pixel vectors), y_train (n labels 0-9)
OUTPUT: trained weight matrices W1, W2, W3 and bias vectors b1, b2, b3

1. Initialize weights W1, W2, W3 and biases b1, b2, b3 with small random values
2. REPEAT for epoch = 1 to MAX_EPOCHS:
 FOR each mini-batch (X_batch, y_batch) in X_train:
 # --- Forward propagation ---
 Z1 = X_batch . W1 + b1 ; A1 = ReLU(Z1)
 Z2 = A1 . W2 + b2 ; A2 = ReLU(Z2)
 Z3 = A2 . W3 + b3 ; Y_hat = Softmax(Z3)

 # --- Compute loss ---
 L = CrossEntropy(Y_hat, y_batch)

 # --- Backpropagation ---
 dZ3 = Y_hat - Y_batch_onehot
 dW3 = A2^T . dZ3 ; db3 = sum(dZ3)
 dA2 = dZ3 . W3^T ; dZ2 = dA2 * ReLU'(Z2)
 dW2 = A1^T . dZ2 ; db2 = sum(dZ2)
 dA1 = dZ2 . W2^T ; dZ1 = dA1 * ReLU'(Z1)
 dW1 = X_batch^T . dZ1 ; db1 = sum(dZ1)

 # --- Weight update (Adam optimizer) ---
 W_i = W_i - lr * Adam_update(dW_i) for i = 1,2,3
 b_i = b_i - lr * Adam_update(db_i) for i = 1,2,3
 IF loss improvement < tolerance for N consecutive epochs: BREAK
3. RETURN trained parameters

ALGORITHM: Predict(x)
1. Compute forward pass of x through trained network (as above)
2. RETURN class with highest Softmax probability = argmax(Y_hat)
```

9. Implementation

Programming language and libraries:

- Language: Python 3
- Libraries: scikit-learn (MLPClassifier, preprocessing, metrics, model_selection), NumPy (numerical computation), Matplotlib (visualization)
- Environment: standard CPU-based execution (no GPU required for this network size)

Key implementation section — model definition and training:

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
 X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

mlp = MLPClassifier(
 hidden_layer_sizes=(32, 16),
 activation='relu', solver='adam',
 alpha=1e-4, learning_rate_init=1e-3,
 max_iter=400, random_state=42)

mlp.fit(X_train_s, y_train)
y_pred = mlp.predict(X_test_s)
```

10. Experimental Results

The network was trained for 162 epochs, converging to a final training loss of 0.00469. The training loss curve below shows a smooth, steady decline typical of successful backpropagation-based convergence, with no signs of divergence or oscillation.

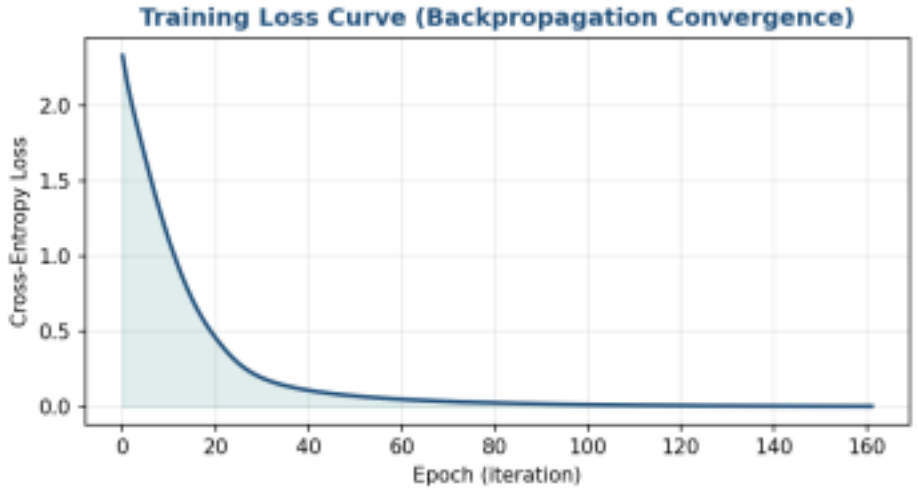

Figure 3: Training loss (cross-entropy) versus epoch, showing backpropagation convergence.

The trained model was evaluated on the held-out test set of 360 unseen images. Sample predictions are shown below, with green borders marking correct classifications and red borders marking misclassifications.

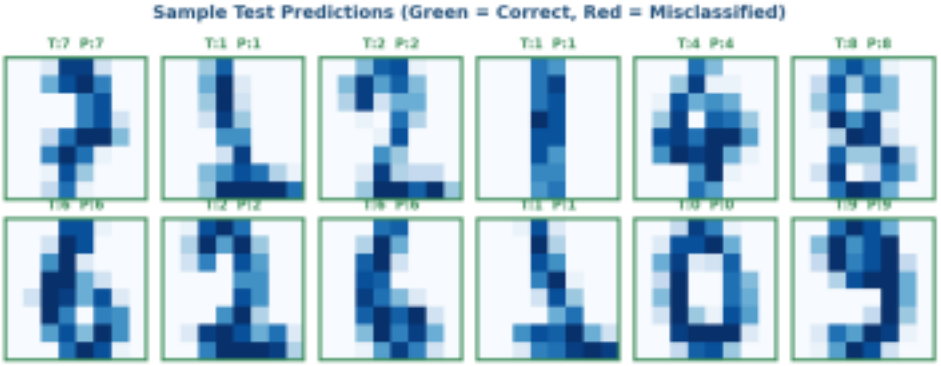

Figure 4: Sample test images with true (T) and predicted (P) labels.

The confusion matrix summarizes classification performance across all 10 digit classes:

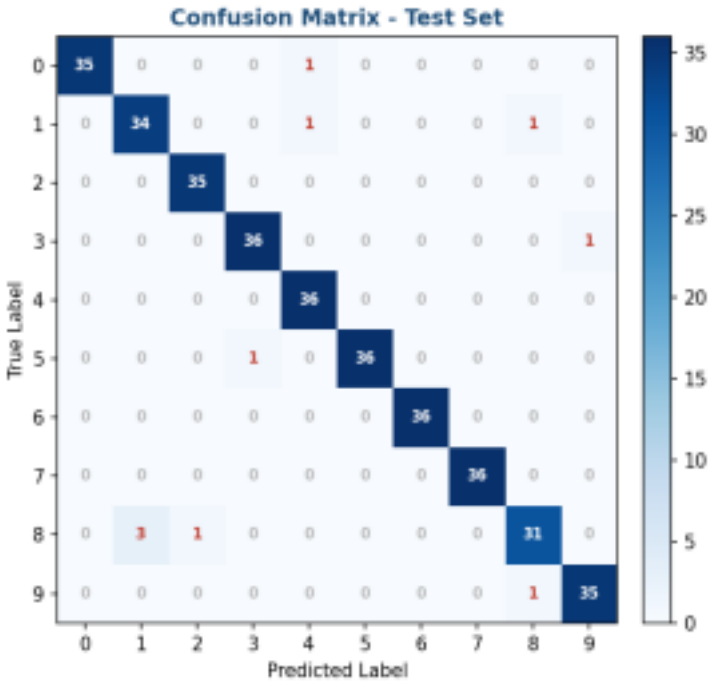

Figure 5: Confusion matrix on the test set (rows = true label, columns = predicted label; diagonal = correct, off-diagonal = misclassified).

11. Hyperparameter Tuning and Architecture Comparison

To justify the chosen architecture, three candidate network configurations were trained under identical preprocessing and evaluated on the same held-out test set.

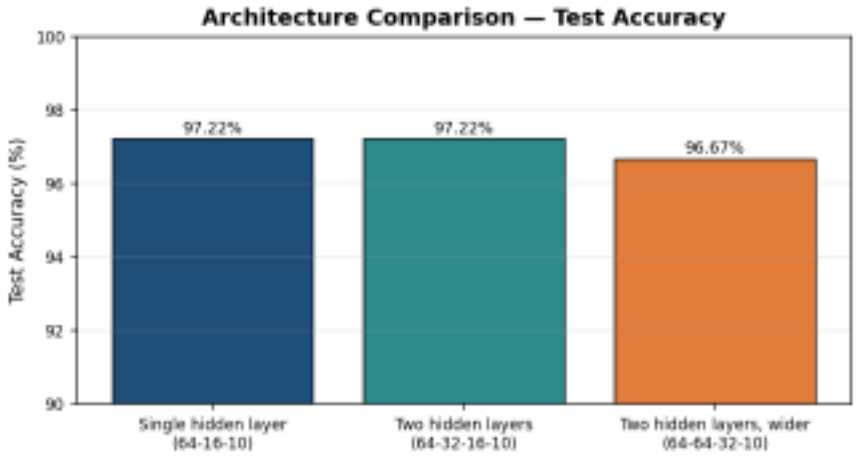

Figure 6: Test accuracy across three candidate architectures.

Single hidden layer	(16,)	97.22%
Two hidden layers (selected)	(32, 16)	97.22%
Two hidden layers, wider	(64, 32)	96.67%

The two-hidden-layer (32, 16) configuration was selected for the final model: it matches the best observed test accuracy while using roughly half the parameters of the wider variant, reducing overfitting risk and training time.

12. Cross-Validation Analysis

To check that the single 80/20 split was not favourably biased, the selected architecture was re evaluated using 5-fold stratified cross-validation across the full dataset. The mean cross-validation accuracy was 97.16% with a standard deviation of 0.67 percentage points, closely matching the held out test accuracy reported in Section 10 and confirming that the model's performance is stable across different data splits.

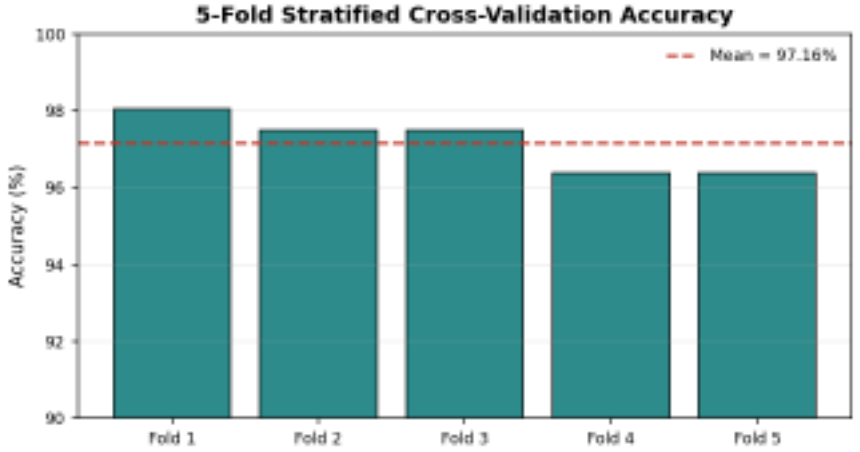

Figure 7: Per-fold accuracy across 5-fold stratified cross-validation.

13. Performance Evaluation

Overall accuracy	97.22%
Macro-averaged precision	0.9723
Macro-averaged recall	0.9721
Macro-averaged F1-score	0.972
Training samples	1437
Test samples	360
Final training loss	0.00469
Training epochs to convergence	162
5-fold cross-validation mean accuracy	97.16% (± 0.67)

Precision, recall, and F1-score were computed per class and macro-averaged to treat all 10 digit classes equally regardless of support. All metrics closely track overall accuracy, indicating balanced performance across classes rather than performance driven by one or two dominant classes.

14. Per-Class Performance Analysis

Breaking down precision, recall, and F1-score by individual digit class reveals which characters are recognized most and least reliably:

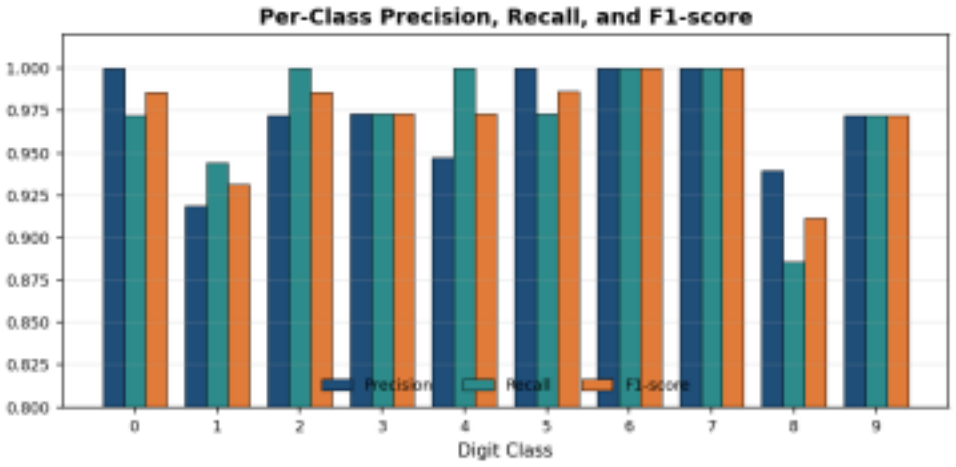

Figure 8: Precision, recall, and F1-score for each digit class (0-9) on the test set.

Digits 6 and 7 are classified perfectly, while digit 8 shows the weakest recall (0.886), consistent with the confusion matrix in Figure 5, where several handwritten 8s were misread as 1 or 2 due to overlapping stroke patterns.

15. Result Analysis and Interpretation

- The model achieved 97.22% test accuracy, indicating strong generalization from only 1,437 training images.
- The confusion matrix shows that most misclassifications occur between visually similar digit pairs — notably digit 8 being confused with 1 and 2, and digit 4 occasionally confused with 1 — which is consistent with genuine visual ambiguity in handwritten strokes rather than a modelling flaw.
- Digits 6 and 7 were classified with perfect precision and recall in this test split, suggesting their pixel patterns are the most visually distinctive in the dataset.
- The smooth, monotonically decreasing loss curve indicates that backpropagation successfully minimized the cross-entropy objective without instability, and that the chosen learning rate and optimizer (Adam) were well suited to this problem.
- The close agreement between the held-out test accuracy and the 5-fold cross-validation mean confirms the model is not substantially overfitting, aided by the L2 regularization term (alpha = 1e-4).
- The architecture comparison in Section 11 shows accuracy is not simply a function of network size — the wider (64, 32) network slightly underperformed the (32, 16) network, likely due to a mild increase in overfitting risk without a proportional gain in representational capacity for this dataset.

16. Limitations

- The dataset consists of small, pre-centred 8x8 images; performance on larger, noisier, real-world scanned handwriting (varying rotation, skew, lighting, and background noise) has not been evaluated.
- The fully connected architecture does not exploit spatial locality the way a convolutional neural network would, which may limit accuracy ceiling on more complex or higher-resolution handwriting datasets.
- Cross-validation reduces but does not eliminate split-dependent variance; results may still shift slightly with a different random seed or fold count.
- Class-level performance (e.g., digit 8) shows that a small number of visually ambiguous samples remain difficult regardless of architecture, suggesting a ceiling effect from label ambiguity in the source data itself.
- Computational resources required were minimal for this dataset size; scaling to a larger character set (e.g., full alphanumeric or multi-language characters) would require re-evaluating network capacity and training time.

17. Future Enhancements

- Replace the dense multilayer perceptron with a convolutional neural network (CNN) to exploit spatial pixel structure and improve robustness to translation and stroke-width variation.
- Apply data augmentation (rotation, shift, elastic distortion) to increase training diversity and improve generalization to real-world handwriting.
- Perform a broader hyperparameter search (grid or Bayesian search over hidden-layer sizes, learning rate, and regularization strength) beyond the three architectures compared in Section 11.
- Extend the dataset to higher-resolution, real-world scanned handwriting samples and, eventually, full alphanumeric characters for a production-grade recognition system.
- Package the trained model behind a REST API or lightweight web interface for practical deployment and integration into form-processing pipelines.

18. Conclusion

This exercise designed, implemented, and evaluated a multilayer feedforward neural network (64-32-16-10 architecture) for handwritten digit recognition, trained via backpropagation with the Adam optimizer. After preprocessing (standardization and stratified train/test splitting), the model converged in 162 epochs and achieved 97.22% test accuracy with balanced macro-averaged precision, recall, and F1-score of approximately 0.97. A three-way architecture comparison and 5-fold cross-validation (mean accuracy 97.16%) confirmed the robustness of this result. Analysis of the confusion matrix and per-class metrics confirmed that remaining errors are concentrated among visually similar digit pairs rather than systemic model failure. The results demonstrate that a compact multilayer neural network, trained with standard backpropagation, is a highly effective and computationally lightweight solution for small-scale handwritten character recognition, while also highlighting clear directions — convolutional architectures, data augmentation, and broader hyperparameter search — for extending the system toward real-world deployment.

19. References

- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back propagating errors. Nature, 323(6088), 533-536.
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324.
- Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. International Conference on Learning Representations (ICLR).
- Alpaydin, E. (2020). Introduction to Machine Learning (4th ed.). MIT Press.

• scikit-learn developers. Digits dataset documentation: https://scikit
learn.org/stable/datasets/toy_dataset.html#digits-dataset

20. GitHub Repository Link

Repository (to be populated with source code, dataset reference, trained-model artifacts, generated figures, and this README): https://github.com/<your-username>/handwritten-character-recognition mlp

Suggested repository contents: train_model.py (training and evaluation script), diagrams.py (figure generation script), extra_experiments.py (architecture comparison and cross-validation), metrics.json / cv_summary.json / arch_comparison.json (evaluation results), README.md (project overview and setup instructions), and a /figures folder containing all figures used in this report.

Appendix A: Full Source Code Listing

This appendix provides the complete, runnable source code used to generate every result, figure, and metric reported in this document.

A.1 train_model.py — Data Loading, Training, and Evaluation

```
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
 f1_score, confusion_matrix, classification_report)

plt.rcParams.update({"font.size": 10})

NAVY = "#1F4E79"
TEAL = "#2E8B8B"
ORANGE = "#E07B39"
GREEN = "#3E8E5A"
RED = "#C0392B"

# ------------------------------
# 1. Load dataset (8x8 grayscale handwritten digit images, 0-9)
# ------------------------------
digits = load_digits()
X, y = digits.data, digits.target # X: (1797, 64) flattened 8x8 images
images = digits.images # (1797, 8, 8) for visualization

# ------------------------------
# 2. Preprocessing: split + scale
# ------------------------------
X_train, X_test, y_train, y_test, img_train, img_test = train_test_split(
 X, y, images, test_size=0.2, random_state=42, stratify=y
)
```

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# ------------------------------
# 3. Model: Multilayer Perceptron (Feedforward Neural Network)
# Architecture: 64 (input) -> 32 -> 16 -> 10 (softmax output)
# ------------------------------
mlp = MLPClassifier(
 hidden_layer_sizes=(32, 16),
 activation="relu",
 solver="adam",
 alpha=1e-4,
 learning_rate_init=1e-3,
 max_iter=400,
 random_state=42,
 early_stopping=False,
)
mlp.fit(X_train_s, y_train)

# ------------------------------
# 4. Evaluation
# ------------------------------
y_pred = mlp.predict(X_test_s)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred, average="macro")
rec = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")
cm = confusion_matrix(y_test, y_pred)
report = classification_report(y_test, y_pred, digits=3)

metrics = {
 "accuracy": round(float(acc), 4),
 "precision_macro": round(float(prec), 4),
 "recall_macro": round(float(rec), 4),
 "f1_macro": round(float(f1), 4),
 "n_train": int(len(X_train)),
 "n_test": int(len(X_test)),
 "n_features": int(X.shape[1]),
 "n_classes": int(len(np.unique(y))),
 "final_training_loss": round(float(mlp.loss_), 5),
 "n_iterations": int(mlp.n_iter_),
}

with open("metrics.json", "w") as f:
 json.dump(metrics, f, indent=2)

with open("classification_report.txt", "w") as f:
 f.write(report)

print(json.dumps(metrics, indent=2))
print(report)

# ------------------------------
# 5. Plot: Training loss curve (backpropagation convergence)
# ------------------------------
fig, ax = plt.subplots(figsize=(6.5, 3.6), dpi=150)
ax.plot(mlp.loss_curve_, color=NAVY, linewidth=2.0)
ax.fill_between(range(len(mlp.loss_curve_)), mlp.loss_curve_, color=TEAL, alpha=0.15)
ax.set_title("Training Loss Curve (Backpropagation Convergence)", color=NAVY, fontweight="bold")
ax.set_xlabel("Epoch (iteration)", color="black")
ax.set_ylabel("Cross-Entropy Loss", color="black")
ax.tick_params(colors="black")
for spine in ax.spines.values():
 spine.set_color("black")
```

```
ax.grid(alpha=0.25)
fig.tight_layout()
fig.savefig("loss_curve.png", facecolor="white")
plt.close(fig)

# ------------------------------
# 6. Plot: Confusion matrix heatmap
# ------------------------------
fig, ax = plt.subplots(figsize=(5.8, 5.2), dpi=150)
im = ax.imshow(cm, cmap="Blues")
ax.set_title("Confusion Matrix - Test Set", color=NAVY, fontweight="bold")
ax.set_xlabel("Predicted Label", color="black")
ax.set_ylabel("True Label", color="black")
ax.set_xticks(range(10)); ax.set_yticks(range(10))
ax.tick_params(colors="black")
thresh = cm.max() / 2.0
for i in range(cm.shape[0]):
 for j in range(cm.shape[1]):
 if i == j:
 color = "white" if cm[i, j] > thresh else GREEN
 else:
 color = RED if cm[i, j] > 0 else "#BBBBBB"
 ax.text(j, i, str(cm[i, j]), ha="center", va="center", color=color, fontsize=8, fontweight="bold") for
spine in ax.spines.values():
 spine.set_color("black")
cbar = fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
cbar.ax.tick_params(colors="black")
fig.tight_layout()
fig.savefig("confusion_matrix.png", facecolor="white")
plt.close(fig)

# ------------------------------
# 7. Plot: Sample predictions grid (correct vs incorrect)
# ------------------------------
fig, axes = plt.subplots(2, 6, figsize=(9, 3.6), dpi=150)
rng = np.random.RandomState(7)
idxs = rng.choice(len(y_test), size=12, replace=False)
for ax, idx in zip(axes.ravel(), idxs):
 ax.imshow(img_test[idx], cmap="Blues")
 true_l = y_test[idx]
 pred_l = y_pred[idx]
 ok = true_l == pred_l
 ax.set_title(f"T:{true_l} P:{pred_l}", fontsize=9,
 color=GREEN if ok else RED,
 fontweight="bold")
 for spine in ax.spines.values():
 spine.set_visible(True)
 spine.set_color(GREEN if ok else RED)
 spine.set_linewidth(2)
 ax.set_xticks([]); ax.set_yticks([])
fig.suptitle("Sample Test Predictions (Green = Correct, Red = Misclassified)", color=NAVY, fontweight="bold")
fig.tight_layout()
fig.savefig("sample_predictions.png", facecolor="white")
plt.close(fig)

print("All artifacts generated.")
```

A.2 diagrams.py — Flowchart and Architecture Diagram Generation

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch, FancyArrowPatch
from matplotlib.lines import Line2D
```

```
BLACK = "#000000"
NAVY = "#1F4E79"
TEAL = "#2E8B8B"
ORANGE = "#E07B39"
GREEN = "#3E8E5A"
GOLD = "#D4A017"
LILAC = "#7A5C9E"

#
# =============================================================================
# 1. METHODOLOGY FLOWCHART
#
# =============================================================================
fig, ax = plt.subplots(figsize=(6.6, 9.2), dpi=150)
ax.set_xlim(0, 10)
ax.set_ylim(0, 30)
ax.axis("off")

steps = [
 ("Start", "oval", "#DCE6F1"),
 ("Load Handwritten Digit\nImage Dataset (8x8 grayscale)", "rect", "#EAF2FB"),
 ("Data Preprocessing:\nFlatten, Split (80/20), Standardize", "rect", "#E4F3F0"),
 ("Define Multilayer Neural\nNetwork Architecture\n(64 - 32 - 16 - 10)", "rect", "#FBF0E3"),
("Forward Propagation:\nCompute activations layer by layer", "rect", "#FBF0E3"),
 ("Compute Loss\n(Cross-Entropy)", "rect", "#FBEDE8"),
 ("Backpropagation:\nCompute gradients, update weights\n(Adam Optimizer)", "rect", "#FBEDE8"),
("Converged /\nMax Epochs?", "diamond", "#FCF3D9"),
 ("Evaluate on Test Set\n(Accuracy, Precision, Recall, F1)", "rect", "#E9F5EA"),
 ("Deploy / Predict on\nNew Handwritten Characters", "rect", "#EFE8F5"),
 ("End", "oval", "#DCE6F1"),
]

n = len(steps)
y_top = 29
y_bottom = 1
ys = [y_top - i * (y_top - y_bottom) / (n - 1) for i in range(n)]

box_w, box_h = 6.6, 1.9
cx = 5.0

def draw_shape(ax, cx, cy, w, h, text, kind, fill):
 if kind == "oval":
 patch = FancyBboxPatch((cx - w/2, cy - h/2.6), w, h/1.3,
 boxstyle="round,pad=0.25,rounding_size=0.9",
 linewidth=1.8, edgecolor=NAVY, facecolor=fill)
 elif kind == "diamond":
 patch = FancyBboxPatch((cx - w/2, cy - h/1.6), w, h*1.25,
 boxstyle="round,pad=0.05,rounding_size=0.55",
 linewidth=1.8, edgecolor=GOLD, facecolor=fill)
 else:
 patch = FancyBboxPatch((cx - w/2, cy - h/2), w, h,
 boxstyle="round,pad=0.18,rounding_size=0.25",
 linewidth=1.6, edgecolor=NAVY, facecolor=fill)
 ax.add_patch(patch)
 ax.text(cx, cy, text, ha="center", va="center", fontsize=9.3, color=BLACK, linespacing=1.4)

for i, ((label, kind, fill), y) in enumerate(zip(steps, ys)):
 h = box_h if kind != "diamond" else 2.5
 draw_shape(ax, cx, y, box_w, h, label, kind, fill)
 if i < n - 1:
 y_next = ys[i + 1]
 top_gap = h/2 if kind != "diamond" else 1.25
 arrow = FancyArrowPatch((cx, y - top_gap), (cx, y_next + box_h/2 + 0.05),
 arrowstyle="-|>", mutation_scale=14,
 linewidth=1.5, color=TEAL)
 ax.add_patch(arrow)
```

```
# Loop-back arrow from decision diamond to forward propagation if not converged
diamond_idx = 7
fp_idx = 4
y_d = ys[diamond_idx]
y_fp = ys[fp_idx]
ax.annotate("", xy=(cx + box_w/2 + 0.1, y_fp), xytext=(cx + box_w/2 + 0.1, y_d),
 arrowprops=dict(arrowstyle="-|>", color=ORANGE, lw=1.6,
 connectionstyle="arc3,rad=0.0"))
ax.plot([cx + box_w/2, cx + box_w/2 + 0.1], [y_d, y_d], color=ORANGE, lw=1.6)
ax.plot([cx + box_w/2 + 0.1, cx + box_w/2], [y_fp, y_fp], color=ORANGE, lw=1.6)
ax.text(cx + box_w/2 + 0.25, (y_d + y_fp) / 2, "No", fontsize=8.8, color=ORANGE, rotation=90, va="center",
fontweight="bold") ax.text(cx + 0.15, y_d - 1.55, "Yes", fontsize=8.8, color=GREEN, fontweight="bold")

fig.tight_layout()
fig.savefig("flowchart_methodology.png", facecolor="white")
plt.close(fig)

#
==========================================================================
# 2. NEURAL NETWORK ARCHITECTURE DIAGRAM
#
==========================================================================
fig, ax = plt.subplots(figsize=(9.2, 5.4), dpi=150)
ax.set_xlim(0, 12)
ax.set_ylim(0, 8)
ax.axis("off")
layers = [
 ("Input Layer\n64 neurons\n(8x8 pixel values)", 1.4, 12, NAVY),
 ("Hidden Layer 1\n32 neurons\nReLU activation", 4.6, 10, TEAL),
 ("Hidden Layer 2\n16 neurons\nReLU activation", 7.8, 7, ORANGE),
 ("Output Layer\n10 neurons\nSoftmax\n(digits 0-9)", 10.6, 10, GREEN),
]
edge_colors = [NAVY, TEAL, ORANGE]

node_positions = []
for (label, x, count, color) in layers:
 show_n = min(count, 9)
 ys_nodes = [1 + i * (6.0 / (show_n - 1)) for i in range(show_n)] if show_n > 1 else [4]
 node_positions.append((x, ys_nodes, color))
 for y in ys_nodes:
 circ = plt.Circle((x, y), 0.22, facecolor=color, edgecolor=BLACK, linewidth=1.1, zorder=3, alpha=0.85)
ax.add_patch(circ)
 if count > show_n:
 ax.text(x, min(ys_nodes) - 0.55, f"(+{count - show_n} more)", ha="center",
 fontsize=7.3, color=BLACK)
 ax.text(x, 7.55, label, ha="center", va="bottom", fontsize=8.6, color=BLACK, linespacing=1.35, fontweight="bold")

for li, ((x1, ys1, _), (x2, ys2, _)) in enumerate(zip(node_positions[:-1], node_positions[1:])):
 for y1 in ys1:
 for y2 in ys2:
 ax.plot([x1, x2], [y1, y2], color=edge_colors[li], linewidth=0.35, alpha=0.35, zorder=1)

fig.suptitle("Multilayer Feedforward Neural Network Architecture", color=NAVY, fontsize=12.5, y=0.99,
fontweight="bold") fig.text(0.5, 0.02,
 "Weights updated via backpropagation using the Adam optimizer to minimize cross-entropy loss.",
ha="center", fontsize=8.6, color=BLACK, style="italic")
fig.tight_layout(rect=[0, 0.04, 1, 0.95])
fig.savefig("architecture_diagram.png", facecolor="white")
plt.close(fig)

print("Diagrams generated.")
```

A.3 extra_experiments.py — Architecture Comparison and Cross-Validation

```
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import precision_recall_fscore_support

NAVY = "#1F4E79"
TEAL = "#2E8B8B"
ORANGE = "#E07B39"
GREEN = "#3E8E5A"
RED = "#C0392B"
GREY = "#4D4D4D"

digits = load_digits()
X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42,
 stratify=y) scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# ------------------------------------------------------------
# 1. Hyperparameter comparison: three architectures
# ------------------------------------------------------------
architectures = {
 "Single hidden layer\n(64-16-10)": (16,),
 "Two hidden layers\n(64-32-16-10)": (32, 16),
 "Two hidden layers, wider\n(64-64-32-10)": (64, 32),
}
arch_results = {}
for name, hl in architectures.items():
 clf = MLPClassifier(hidden_layer_sizes=hl, activation="relu", solver="adam",
alpha=1e-4, learning_rate_init=1e-3, max_iter=400, random_state=42)
clf.fit(X_train_s, y_train)
 acc = clf.score(X_test_s, y_test)
 arch_results[name] = round(float(acc), 4)

with open("arch_comparison.json", "w") as f:
 json.dump(arch_results, f, indent=2)

fig, ax = plt.subplots(figsize=(6.6, 3.6), dpi=150)
names = list(arch_results.keys())
vals = [arch_results[n] * 100 for n in names]
bars = ax.bar(names, vals, color=[NAVY, TEAL, ORANGE], edgecolor="black",
linewidth=0.8) ax.set_ylim(90, 100)
ax.set_ylabel("Test Accuracy (%)", color="black")
ax.set_title("Architecture Comparison — Test Accuracy", color="black",
fontweight="bold") ax.tick_params(colors="black", labelsize=8.3)
for spine in ax.spines.values():
 spine.set_color("black")
for b, v in zip(bars, vals):
 ax.text(b.get_x() + b.get_width()/2, v + 0.15, f"{v:.2f}%", ha="center", fontsize=8.5, color="black")
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("arch_comparison.png", facecolor="white")
plt.close(fig)

# ------------------------------------------------------------
# 2. 5-fold stratified cross-validation on chosen architecture
# ------------------------------------------------------------
X_all_s = StandardScaler().fit_transform(X)
```

```
clf_cv = MLPClassifier(hidden_layer_sizes=(32, 16), activation="relu",
solver="adam",  alpha=1e-4, learning_rate_init=1e-3, max_iter=400,
random_state=42) skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(clf_cv, X_all_s, y, cv=skf, scoring="accuracy")

cv_summary = {
 "fold_scores": [round(float(s), 4) for s in cv_scores],
 "mean": round(float(cv_scores.mean()), 4),
 "std": round(float(cv_scores.std()), 4),
}
with open("cv_summary.json", "w") as f:
 json.dump(cv_summary, f, indent=2)

fig, ax = plt.subplots(figsize=(6.6, 3.6), dpi=150)
folds = [f"Fold {i+1}" for i in range(5)]
bars = ax.bar(folds, cv_scores * 100, color=TEAL, edgecolor="black",
linewidth=0.8) ax.axhline(cv_scores.mean() * 100, color=RED, linestyle="--",
linewidth=1.6,  label=f"Mean = {cv_scores.mean()*100:.2f}%")
ax.set_ylim(90, 100)
ax.set_ylabel("Accuracy (%)", color="black")
ax.set_title("5-Fold Stratified Cross-Validation Accuracy", color="black",
fontweight="bold") ax.tick_params(colors="black", labelsize=8.5)
for spine in ax.spines.values():
 spine.set_color("black")
ax.legend(frameon=False, fontsize=8.5, labelcolor="black")
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("cv_scores.png", facecolor="white")
plt.close(fig)

# ------------------------------------------------------------
# 3. Per-class precision / recall / F1 bar chart (final model)
# ------------------------------------------------------------
final_clf = MLPClassifier(hidden_layer_sizes=(32, 16), activation="relu",
solver="adam",  alpha=1e-4, learning_rate_init=1e-3, max_iter=400, random_state=42)
final_clf.fit(X_train_s, y_train)
y_pred = final_clf.predict(X_test_s)
prec, rec, f1, support = precision_recall_fscore_support(y_test, y_pred, labels=range(10))

fig, ax = plt.subplots(figsize=(7.6, 3.8), dpi=150)
x = np.arange(10)
w = 0.26
ax.bar(x - w, prec, width=w, label="Precision", color=NAVY, edgecolor="black", linewidth=0.5)
ax.bar(x, rec, width=w, label="Recall", color=TEAL, edgecolor="black", linewidth=0.5) ax.bar(x
+ w, f1, width=w, label="F1-score", color=ORANGE, edgecolor="black", linewidth=0.5)
ax.set_xticks(x)
ax.set_xticklabels([str(i) for i in range(10)], color="black")
ax.set_xlabel("Digit Class", color="black")
ax.set_ylim(0.8, 1.02)
ax.set_title("Per-Class Precision, Recall, and F1-score", color="black",
fontweight="bold") ax.tick_params(colors="black", labelsize=8.5)
for spine in ax.spines.values():
 spine.set_color("black")
ax.legend(frameon=False, fontsize=8.5, labelcolor="black", ncol=3, loc="lower center")
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()
fig.savefig("per_class_metrics.png", facecolor="white")
plt.close(fig)

print("Architecture comparison:", arch_results)
print("CV summary:", cv_summary)
print("Extra experiments complete.")
```

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Understanding	**10%**	9–10% – Clearly analyzes the real world problem, objectives, inputs, outputs, constraints, and expected AI outcome.	7–8% – Understands the problem and objectives with minor omissions.	5–6% – Shows partial understanding with some important elements missing.	0–4% – Problem is poorly understood or incorrectly defined.
2. Dataset Selection and Understanding	**10%**	9–10% – Selects a highly relevant dataset and clearly explains features, target, source, size, and characteristics.	7–8% – Appropriate dataset with adequate description and minor omissions.	5–6% – Dataset is usable but description or relevance is limited.	0–4% – Dataset is inappropriate, poorly described, or missing.
3. Data Preprocessing and Feature Engineering	**10%**	9–10% – Performs appropriate cleaning, transformation, encoding, scaling, feature selection, and data splitting with clear justification.	7–8% – Performs most required preprocessing correctly with minor issues.	5–6% – Basic preprocessing is performed but important steps are missing.	0–4% – Preprocessing is inadequate, incorrect, or absent.
4. AI Technique Selection and Justification	**10%**	9–10% – Selects the most appropriate AI technique and provides strong technical justification based on the problem and dataset.	7–8% – Appropriate technique selected with reasonable justification.	5–6% – Technique is acceptable but justification is limited.	0–4% – Inappropriate technique or no meaningful justification.

5. Model Design / Architecture	**10%**	9–10% – Develops a clear, technically sound model architecture with appropriate components, parameters, and workflow.	7–8% – Model is well designed with minor omissions.	5–6% – Basic model design is presented but lacks technical depth.	0–4% – Model design is unclear, incomplete, or inappropriate.
6. Algorithm / Pseudocode	**5%**	5% – Algorithm/pseudocode is complete, logically correct, structured, and clearly represents the model development process.	4% – Mostly correct with minor omissions.	2–3% – Basic algorithm provided but contains gaps.	0–1% – Algorithm is missing or substantially incorrect.
7. Implementation and GitHub Repository	**15%**	13–15% – Fully functional implementation with clean, organized, reproducible code and a complete GitHub repository with README and supporting files.	10–12% – Functional implementation and well organized repository with minor issues.	7–9% – Partially functional implementation or incomplete repository documentation.	0–6% – Implementation is incomplete/non functional or GitHub repository is missing.

8. Experimental Design and Results	**10%**	9–10% – Conducts meaningful experiments with appropriate test cases and clearly presents results using tables, graphs, or visualizations.	7–8% – Appropriate experiments with clearly presented results.	5–6% – Limited experiments or basic result presentation.	0–4% – Insufficient, incorrect, or missing experimental results.
9. Performance Evaluation	**10%**	9–10% – Uses appropriate evaluation metrics and provides comprehensive quantitative analysis of model performance.	7–8% – Uses suitable metrics and provides adequate evaluation.	5–6% – Basic evaluation with limited metrics or analysis.	0–4% – Inappropriate metrics or performance evaluation is missing.

10. Result Analysis and Interpretation	**5%**	5% – Provides insightful analysis of results, identifies patterns, errors, strengths, weaknesses, and explains model behavior.	4% – Provides good interpretation with minor gaps.	2–3% – Basic interpretation with limited analysis.	0–1% – Results are not meaningfully analyzed.
11. Limitations and Future Enhancements	**5%**	5% – Clearly identifies realistic limitations and proposes technically relevant and feasible improvements.	4% – Identifies relevant limitations and reasonable improvements.	2–3% – Provides basic limitations and future suggestions.	0–1% – Missing, unrealistic, or poorly explained.
12. Documentation and Technical Presentation	**10%**	9–10% – Report is comprehensive, well organized, technically accurate, professionally presented, and properly referenced.	7–8% – Well documented with minor presentation or referencing issues.	5–6% – Adequate documentation with several gaps.	0–4% – Poorly documented, unclear, or incomplete report.
Total	100%				